



Dr. APJ ABDUL KALAM SCHOOL OF ENGINEERING
DEPARTMENT OF ENGINEERING

NAME OF THE STUDENT : NIVEDH SUNIL

ROLL NO.: 24BTCE111

BRANCH: BTCE

COURSE : ADVANCED PYTHON

ASSIGNMENT No: 01

DATE ISSUED: 06/07/2026

DATE OF SUBMISSION: 06/07/2026

SUBMITTED TO: PROF. KAMALADHARAN

Signature Of The Student

Signature Of The Faculty

1. Explain the concept of Computer Vision and discuss its importance in modern applications.

Ans.

- Computer vision is a branch of computer science and artificial intelligence that enables machines to acquire, process, analyze, and understand visual information from images and videos
- While image processing focuses on enhancing or transforming images at the pixel level, computer vision goes a step further by attempting to understand the actual content of the image and make decisions based on that visual data
- By combining concepts from mathematics, programming, machine learning, and artificial intelligence, computer vision systems extract useful information from pixels
- For instance, while increasing the brightness of a dark photo is image processing, identifying whether that photo contains a traffic sign, a vehicle, or a person is a computer vision task

Importance in Modern Applications

- Visual information is one of the richest sources of data in the real world, and computer vision attempts to give machines the same natural visual understanding that humans possess
- The field has become increasingly vital due to the widespread availability of smartphones, digital cameras, drones, GPUs, cloud computing, and deep learning models
- The primary importance of computer vision lies in its ability to improve speed, accuracy, and consistency in tasks requiring visual inspection
- Because a trained system can examine thousands of images faster than a human, it can automate or assist in tasks that used to be strictly manual

Applications

- Healthcare: It assists doctors by analyzing medical images, detecting tumors, and supporting X-ray and MRI diagnostics, which improves overall screening efficiency
- Automotive: It is foundational to autonomous vehicles and driver assistance systems, enabling tasks like lane detection, pedestrian detection, and traffic sign recognition
- Agriculture: It improves precision farming and yield monitoring through fruit counting, weed identification, and crop disease detection
- Manufacturing: It reduces manual inspection time and improves quality control by automatically detecting product defects and reading barcodes
- Security & Surveillance: It enhances monitoring in public and private spaces through face detection, intrusion detection, and person tracking
- Robotics: It allows robots to safely interact with real-world environments by facilitating navigation, obstacle avoidance, and object localization
- Banking & Retail: It reduces errors and automates workflows through identity verification, Optical Character Recognition (OCR) for cheque processing, shelf monitoring, and customer analytics

2. Describe the OpenCV library, its architecture, features, capabilities, and advantages.

Ans.

- OpenCV (Open Source Computer Vision Library) is a highly popular, open-source collection of optimized algorithms used for image processing, computer vision, and machine learning
- It was designed to provide a common infrastructure for computer vision applications, allowing developers to use tested, ready-made functions rather than building algorithms from scratch
- It is widely used for building projects such as real-time camera applications, face detection systems, and traffic monitoring systems

Architecture and Major Modules

- core: Handles basic mathematical operations, data structures (like arrays and matrices), data types, and performance utilities
- imgcodecs: Responsible for reading and writing image files, such as `cv.imread()` and `cv.imwrite()`
- highgui: Used for displaying images and creating simple graphical windows using functions like `cv.imshow()`

- **imgproc:** Contains essential image processing operations, such as resizing, cropping, filtering, edge detection, and colour conversion
- **videoio:** Manages the reading and writing of video streams, including webcam capture and frame extraction
- **features2d:** Dedicated to feature detection and description, such as finding corners, keypoints, and handling SIFT-like workflows
- **objdetect:** Provides utilities for object detection, such as cascade classifiers and QR code detection
- **dnn:** Supports deep neural network inference, allowing users to run trained models for classification or object detection

Features and Capabilities

- **Cross-Platform and Multi-Language Support:** OpenCV runs on Windows, macOS, Linux, and Android, and provides interfaces for Python, C++, and Java
- **Image Input/Output and Transformation:** It can read, save, display, resize, crop, rotate, flip, and warp images
- **Colour Processing:** It allows seamless conversion between different colour models, such as BGR, RGB, grayscale, and HSV
- **Filtering and Enhancement:** It can adjust image contrast, denoise low-light images, blur, and sharpen visuals
- **Object and Feature Detection:** It can detect shapes, faces, and regions of interest, as well as find critical edges and corners to match objects across different images
- **Video and Real-Time Processing:** It is optimized for real-time vision applications, allowing it to easily capture and process frames from live webcams or video files
- **Machine Learning and DNN:** It can run machine learning algorithms and perform inference using trained deep learning networks
- **Camera Calibration:** It can estimate camera parameters to correct lens distortion, which is critical for accuracy in robotics and industrial inspection

Advantages

- **Fast and Optimized:** It is highly optimized for complex image processing tasks, making it suitable for real-time applications
- **Comprehensive:** It consolidates a massive number of algorithms and functions into a single library
- **NumPy Integration:** In Python, OpenCV represents images as NumPy arrays, making image data incredibly easy to inspect and manipulate using standard array indexing and slicing
- **Excellent for Prototyping:** It is an ideal tool for rapid project development and prototyping
- **Strong Educational Value:** Because of its large community, extensive documentation, and open-source nature, it is highly preferred for learning
- **It bridges the gap between theory and practice,** allowing students to immediately see the visual results of their code

3. Explain digital image representation, including pixels, image resolution, and image dimensions.

Ans.

Digital Image Representation:

- **Pixels** The fundamental building block of a digital image is the pixel (short for "picture element")
- A pixel is the smallest addressable and controllable unit within an image
- When you zoom closely into an image, these pixels become visible as individual square-like blocks of color
- Each pixel stores numerical values that describe intensity or color
- For example, in a standard 8-bit grayscale image, a pixel's value ranges from 0 to 255. In this scale, 0 represents pure black, 255 represents pure white, and the numbers in between represent varying shades of gray
- In a color image, each pixel usually contains three separate values (one for each color channel)
- **Image Resolution** Image resolution refers to the total number of pixels contained within an image, and it is usually expressed as width x height
- For example, a "Full HD" image with a resolution of 1920 x 1080 means it has 1920 pixels in each horizontal row and 1080 pixels in each vertical column
- Multiplying these numbers (1920 x 1080) reveals the total pixel count, which is over 2 million pixels

- While higher-resolution images capture much finer detail, they also require significantly more storage space, memory, and computational power
- To manage this computational cost, computer vision pipelines often resize images to smaller, standardized dimensions before processing them
- Image Dimensions and Matrix Representation In programming environments like Python with OpenCV, digital images are represented as multidimensional NumPy arrays
- The dimensions (or "shape") of this array depend on the type of image:
- Grayscale Images: These are represented as a two-dimensional array with the shape (height, width)
- Each pixel has only one channel (intensity)

Color Images:

- These are represented as a three-dimensional array with the shape (height, width, channels)
- The first dimension represents the rows (height), the second represents the columns (width), and the third represents the color channels (depth)
- A standard color image has three channels, typically corresponding to Red, Green, and Blue (though OpenCV loads them in Blue, Green, Red order by default)
- Images with Transparency: These add a fourth "alpha" channel to represent transparency, making the shape (height, width, 4)

4. Compare the RGB, BGR, Grayscale, HSV, and CMYK color models with suitable applications.

Ans.

Grayscale

- Components: This model consists of only a single channel representing intensity
- In a standard 8-bit image, values range from 0 (pure black) to 255 (pure white)
- Suitable Applications: Because it reduces a 3-channel color image to a single channel, it requires less memory and computational power
- It is widely used for edge detection, image thresholding, document processing, and simple measurements

RGB (Red, Green, Blue)

- Components: This is a three-channel model that mixes red, green, and blue light to create colors
- Suitable Applications: RGB is the standard display model used by hardware like computer monitors and digital cameras, as well as many image processing and plotting libraries, such as Matplotlib

BGR (Blue, Green, Red)

- Components: This contains the exact same three color components as RGB, but the channel order is reversed
- Suitable Applications: BGR is primarily important because it is the default channel order used by OpenCV when reading and processing color images
- If you load a BGR image in OpenCV and attempt to display it using a library that expects RGB (like Matplotlib), the colors will appear incorrect unless you convert the image first.

HSV (Hue, Saturation, Value)

- Components: Instead of mixing primary colors, this model separates the image into Hue (the actual color), Saturation (the intensity or purity of the color), and Value (the brightness)
- Suitable Applications: Because it isolates the color information (Hue) from the lighting conditions (Value), it is highly effective for color segmentation, detecting specific colored objects (such as finding a red object), and object tracking

CMYK (Cyan, Magenta, Yellow, Black)

- Components: This is a four-component subtractive color model
- Suitable Applications: Unlike RGB which is used for digital screens, CMYK is specifically designed for printing and publishing workflows

5. Write a Python program using OpenCV to read, display, and save an image. Explain the functions used.

Ans.

```
import cv2

image = cv2.imread('input.jpg')

if image is None:
    print("Error: Could not read the image. Check the file path.")
else:
    cv2.imshow('Display Window', image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
    cv2.imwrite('output.jpg', image)
    print("Image saved as output.jpg")
```

Explanation of Functions Used

`cv2.imread(path, flag)`

Reads an image from the given file path and returns it as a NumPy array (height x width x channels). By default, images are loaded in BGR color order, not RGB. Optional flags:

`cv2.IMREAD_COLOR` (default) - loads a color image, ignoring transparency

`cv2.IMREAD_GRAYSCALE` - loads image as grayscale

`cv2.IMREAD_UNCHANGED` - loads image with alpha channel if present

If the path is wrong or the file is unreadable, it returns None instead of raising an error, which is why checking if image is None is good practice.

`cv2.imshow(window_name, image)`

Opens a window with the given title and displays the image array in it. It needs to be paired with `cv2.waitKey`, otherwise the window will flash and close immediately, or not render at all in some environments.

`cv2.waitKey(delay)`

Pauses program execution and waits for a keyboard event.

`waitKey(0)` waits indefinitely until any key is pressed.

`waitKey(n)` waits for n milliseconds, useful for video frames or timed displays.

It also returns the key code, which can be used for interactive control, for example, press q to quit.

`cv2.destroyAllWindows()`

Closes all OpenCV-created windows and frees associated memory. Good practice to call this after you are done displaying images.

`cv2.imwrite(filename, image)`

Saves the image array to disk in the format specified by the file extension, such as .jpg, .png, .bmp. Returns True if the save was successful, False otherwise. Compression or quality can be controlled via extra parameters, for example:

`cv2.imwrite('output.jpg', image, [cv2.IMWRITE_JPEG_QUALITY, 90])`

6. Develop a Python program to perform image manipulation operations such as resizing, rotating, flipping, and cropping.

Ans.

```
import cv2

image = cv2.imread('input.jpg')

if image is None:
    print("Error: Could not read the image. Check the file path.")
else:
    # 1. Resizing
    resized = cv2.resize(image, (300, 300))
    cv2.imwrite('resized.jpg', resized)

    # 2. Rotating
    (h, w) = image.shape[:2]
    center = (w // 2, h // 2)
    rotation_matrix = cv2.getRotationMatrix2D(center, 45, 1.0)
    rotated = cv2.warpAffine(image, rotation_matrix, (w, h))
    cv2.imwrite('rotated.jpg', rotated)

    # 3. Flipping
    flipped_horizontal = cv2.flip(image, 1)
    flipped_vertical = cv2.flip(image, 0)
    flipped_both = cv2.flip(image, -1)
    cv2.imwrite('flipped_horizontal.jpg', flipped_horizontal)
    cv2.imwrite('flipped_vertical.jpg', flipped_vertical)
    cv2.imwrite('flipped_both.jpg', flipped_both)

    # 4. Cropping
    cropped = image[50:200, 100:300]
    cv2.imwrite('cropped.jpg', cropped)

    print("All operations completed and images saved.")
```

7. Write a Python program to display image properties such as dimensions, channels, data type, and total number of pixels.

Ans.

```
import cv2

image = cv2.imread('input.jpg')
```

```

if image is None:
    print("Error: Could not read the image. Check the file path.")
else:
    height, width, channels = image.shape
    data_type = image.dtype
    total_pixels = image.size
    print("Image Properties")
    print("Height:", height)
    print("Width:", width)
    print("Number of Channels:", channels)
    print("Data Type:", data_type)
    print("Total Number of Pixels:", total_pixels)

```

8. Analyze the impact of image resolution on image quality, memory usage, and processing time in computer vision applications.

Ans.

Image resolution, which refers to the total number of pixels contained in an image (usually expressed as width × height), has a profound impact on three critical aspects of computer vision applications: image quality, memory usage, and processing time.

- **Image Quality and Detail:** Higher resolution images contain significantly more pixels, which allows them to capture much finer visual details. For example, a 4K image (3840 x 2160) contains roughly 8.29 million pixels, whereas a standard VGA image (640 x 480) contains only about 0.31 million pixels. Because the success of a computer vision system depends heavily on the quality of its visual input, a clear and highly detailed image often yields better results for algorithms tasked with identifying edges, corners, and object features.
- **Memory Usage:** The memory required by an image scales directly with its resolution, the number of color channels, and the bytes per channel. For instance, a standard Full HD image (1920 x 1080) contains over 2.07 million pixels. If it is an 8-bit color image containing three channels (like standard BGR or RGB), it will consume approximately 6.22 MB of memory before any data compression is applied. Consequently, as resolution increases, the storage and RAM required to hold the image matrix increase substantially.
- **Processing Time and Computational Cost:** Inside a computer system, an image is processed as a multidimensional numerical array. More pixels inherently mean a larger array, which requires the computer's processor to perform a vastly higher number of calculations. As a result, feeding full high-resolution images directly into an algorithm drastically increases the computational cost and slows down processing time.

9. Develop a Python application using OpenCV that allows users to load an image, view its properties, perform basic image manipulations, and save the processed image.

```

Ans.  import cv2
import os
def load_image(path):
    image = cv2.imread(path)
    if image is None:

```

```
print("Error: Could not load image. Check the file path.")
return image

def show_properties(image):
    height, width, channels = image.shape
    data_type = image.dtype
    total_pixels = image.size
    print("\nImage Properties")
    print("Height:", height)
    print("Width:", width)
    print("Number of Channels:", channels)
    print("Data Type:", data_type)
    print("Total Number of Pixels:", total_pixels)

def resize_image(image):
    w = int(input("Enter new width: "))
    h = int(input("Enter new height: "))
    return cv2.resize(image, (w, h))

def rotate_image(image):
    angle = float(input("Enter rotation angle in degrees: "))
    (h, w) = image.shape[:2]
    center = (w // 2, h // 2)
    matrix = cv2.getRotationMatrix2D(center, angle, 1.0)
    return cv2.warpAffine(image, matrix, (w, h))

def flip_image(image):
    print("1. Horizontal Flip")
    print("2. Vertical Flip")
    print("3. Both")
    choice = input("Enter choice: ")
    if choice == "1":
        return cv2.flip(image, 1)
    elif choice == "2":
        return cv2.flip(image, 0)
    elif choice == "3":
        return cv2.flip(image, -1)
    else:
        print("Invalid choice, returning original image.")
    return image
```



```
def crop_image(image):
    y1 = int(input("Enter start y: "))
    y2 = int(input("Enter end y: "))
    x1 = int(input("Enter start x: "))
    x2 = int(input("Enter end x: "))
    return image[y1:y2, x1:x2]

def save_image(image):
    filename = input("Enter filename to save as (e.g. output.jpg): ")
    cv2.imwrite(filename, image)
    print("Image saved as", filename)

def main():
    path = input("Enter path of image to load: ")
    image = load_image(path)
    if image is None:
        return

    while True:
        print("\nMenu")
        print("1. View Image")
        print("2. View Properties")
        print("3. Resize")
        print("4. Rotate")
        print("5. Flip")
        print("6. Crop")
        print("7. Save Image")
        print("8. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            cv2.imshow("Image", image)
            cv2.waitKey(0)
            cv2.destroyAllWindows()
        elif choice == "2":
            show_properties(image)
        elif choice == "3":
```

```

    image = resize_image(image)
    print("Image resized.")
elif choice == "4":
    image = rotate_image(image)
    print("Image rotated.")
elif choice == "5":
    image = flip_image(image)
    print("Image flipped.")
elif choice == "6":
    image = crop_image(image)
    print("Image cropped.")
elif choice == "7":
    save_image(image)
elif choice == "8":
    print("Exiting application.")
    break
else:
    print("Invalid choice, try again.")
if name == "main":
    main()

```

10. Prepare a case study on a real-world computer vision application and explain how OpenCV is used to implement the solution.

Ans.

Case Study: Automated Face Region Extraction from Student ID Cards

The Real-World Application A common administrative application of computer vision in education and security is the automated processing of identification documents. In this case study, a college wants to build a system that captures images of student ID cards and automatically crops out just the face region so it can be stored in a database or used for further processing (such as identity verification)

- This workflow mirrors the standard computer vision pipeline: acquiring the image, preprocessing it, extracting the relevant features (the region of interest), and outputting a result
- How OpenCV is Used to Implement the Solution To build this system, we can utilize several core modules and fundamental functions within the OpenCV library

step-by-step implementation:

1. Image Acquisition and Reading The first step is to load the captured ID card image from the system's storage.

- OpenCV Implementation: The function `cv.imread()` is used to read the file from the disk
- Best Practice: It is essential to include basic error checking by verifying if the loaded image is `None`. This ensures the program does not crash if the file path is incorrect or the format is unsupported

2. Inspecting Image Properties Before manipulating the image, the system should verify its dimensions to ensure it meets the expected input criteria.

- OpenCV Implementation: By calling `image.shape`, the program can retrieve the height, width, and number of color channels
- This step ensures the image is properly formatted (e.g., a standard color image will have 3 channels) before proceeding

3. Object Detection (Identifying the Face) OpenCV can perform face detection using its `objdetect` module or deep neural networks via the `dnn` module the specific algorithmic code for detecting the face coordinates is not detailed in the sources. However, the system would typically output the bounding box coordinates of the detected face at this stage).

4. Cropping the Region of Interest (ROI) Once the system identifies the coordinates of the student's face on the ID card, it must isolate this specific area to remove the irrelevant background information (like text and logos).

- OpenCV Implementation: Because OpenCV represents images as multidimensional NumPy arrays, cropping does not require a special function; it is performed using array slicing
- The system will use the format `image[y1:y2, x1:x2]`, where the row range (`y1:y2`) represents the vertical boundaries of the face, and the column range (`x1:x2`) represents the horizontal boundaries

5. Resizing the Cropped Image To ensure uniformity in the college's database, the extracted face must be standardized to a specific dimension.

- OpenCV Implementation: The `cv.resize()` function is applied to the cropped image to scale it to a fixed width and height
- This standardization is also a crucial preparation step if the faces are later going to be fed into a machine learning model for recognition

6. Saving the Final Output Finally, the processed, resized face region needs to be saved as a new file.

- OpenCV Implementation: The function `cv.imwrite()` is used to save the image to the disk
- The program should check the boolean return value of this function (which returns `True` upon success) to confirm that the file was successfully written and that there were no folder permission errors
- By chaining these fundamental OpenCV operations together—reading, inspecting, array slicing, resizing, and writing—the college can successfully automate a task that would otherwise require manual image editing